

GameStore — Configuración de Spring Security

Explicación de qué hace cada pieza de configuración de seguridad en la versión microservicios (4 servicios Spring Boot + **api-gateway** + librería **common-security**). Roles de negocio: **ADMIN**, **VENDEDOR**, **COMPRADOR**.

El modelo en una frase

auth-service firma los JWT con una clave privada RSA; **todos los demás sólo validan** con la clave pública. El **gateway** hace una validación de “primera línea” (rechaza rápido lo que no trae token) y **cada servicio vuelve a validar** por su cuenta: defensa en profundidad. No hay sesión de servidor salvo en el flujo de login con Google.

Componente	¿Firma?	¿Valida?	Sesión	Responsabilidad de seguridad
api-gateway	No	Sí (1ª línea)	No (reactivo)	Corta sin token, limpia headers, propaga identidad, CORS
auth-service	Sí	Sí	IF_REQUIRED	Login, BCrypt, refresh tokens, 2FA, logout, Google
usuarios-service	No	Sí	STATELESS	ABM de usuarios sólo ADMIN; API interna
catalogo-service	No	Sí	STATELESS	Reglas por rol + “es tu juego”; API interna
common-security	—	—	—	Librería: JwtService + JwtAuthenticationFilter + InternalTokenFilter

1 · common-security (librería compartida)

services/common-security/src/main/java/com/gamestore/common/security/

La incluye cada servicio como dependencia. Con sólo tenerla en el classpath quedan listos los beans de JWT; después cada servicio engancha el filtro en su propio SecurityFilterChain.

CommonSecurityAutoConfiguration (@AutoConfiguration)

`@EnableConfigurationProperties(JwtProperties)`	Activa el binding de las propiedades app.jwt.*
`@Bean jwtService(...)`	Crea un JwtService con las claves configuradas. @ConditionalOnMissingBean: un servicio puede reemplazarlo.
`@Bean jwtAuthenticationFilter(...)`	Crea el filtro servlet de validación de JWT. @ConditionalOnClass(OncePerRequestFilter): sólo en apps servlet (no en el gateway reactivo, que trae su propia versión).

JwtProperties (prefix = "app.jwt")

`public-key-location`	Clave pública RSA en PEM (X.509). SIEMPRE presente: valida la firma de los tokens entrantes. Ej: classpath:jwt-public.pem o file:/run/keys/jwt-public.pem
`private-key-location`	Clave privada RSA (PKCS#8). SÓLO en auth-service. Si falta, firmar lanza excepción (fail-fast: un servicio que no debe emitir tokens no puede hacerlo por accidente).
`access-expiration-ms`	Vida del access token. Por defecto 900000 = 15 min. La sesión larga la sostiene el refresh token.
`issuer`	Va en el claim iss (gamestore-auth). Todos los validadores lo EXIGEN (requireIssuer).

JwtService

Genera y valida los access token, firmados con **RS256** (RSA) — reemplaza el HMAC compartido del monolito.

- **auth-service** lo construye con privada + pública → puede firmar y validar.
- **gateway / usuarios / catalogo** lo construyen sólo con la pública → validan, no firman.

```
generar(uid, username, rol):
    si privateKey == null -> IllegalStateException (fail-fast)
    Jwts.builder()
        .issuer("gamestore-auth")
        .subject(username)           # claim "sub"
        .claim("uid", uid)           # id numérico: pertenencia sin ir a la BD
        .claim("rol", rol)           # ADMIN | VENDEDOR | COMPRADOR
        .issuedAt(now).expiration(now + 15min)
        .signWith(privateKey, RS256)

validar(token):
    Jwts.parser()
        .verifyWith(publicKey)       # firma
        .requireIssuer("gamestore-auth")
        .parseSignedClaims(token)     # JwtException si corrupto / mal firmado /
                                     # expirado / otro issuer
```

PemKeys

Lee claves RSA en formato PEM (las de keys/generate-keys.sh, openssl) y las convierte en objetos java.security.Key. Sin BouncyCastle: alcanza el JDK. Pública = X.509 SubjectPublicKeyInfo; privada = PKCS#8 ("BEGIN PRIVATE KEY").

JwtAuthenticationFilter (OncePerRequestFilter)

Pone la identidad en el SecurityContext de cada servicio, **sin tocar la base de datos** (todo sale del token, ya firmado por auth-service).

```
si hay header Authorization: Bearer <token> y aún no hay autenticación:
    jws = jwtService.validar(token)
    princ = AuthPrincipal(uid, sub, rol) # desde los claims
    SecurityContext.authentication =
        UsernamePasswordAuthenticationToken(princ, null, ["ROLE_"+rol]) # -> hasRole(...)
si el token falta o es inválido:
    no se autentica -> las reglas del SecurityFilterChain deciden 401 / 403
```

InternalTokenFilter (OncePerRequestFilter)

Protege los endpoints /internal/ **que un servicio expone** sólo para que lo llamen otros servicios **(nunca el navegador)**. Los servicios lo instancian a mano en su SecurityConfig.

```
shouldNotFilter: sólo actúa sobre /internal/**

doFilter:
    si header X-Internal-Token != secreto compartido (app.internal.secret):
        401 {"mensaje":"Llamada interna no autorizada"}
    si coincide:
        SecurityContext.authentication = ... con ROLE_INTERNAL
        -> .requestMatchers("/internal/**").hasRole("INTERNAL") lo deja pasar
```

Nota de diseño. El secreto compartido es deliberadamente simple. En un cluster real se reforzaría con network policies / mTLS y el token sería una segunda barrera. Para la demo (docker-compose en una red interna) el header alcanza.

AuthPrincipal (record)

La identidad que viaja en el JWT y queda como principal en el SecurityContext. No hay entidad JPA detrás: **sólo lo que el token afirma**. Campos: id (claim uid), username (sub), rol. authorities() → ROLE_<rol>; isAdmin() para chequeos de dominio.

2 · api-gateway

services/api-gateway/.../gateway/ · src/main/resources/application.yml

Lo único que el navegador ve. Rutea **/api/ y /uploads/**, sirve el build de React, y hace seguridad de “primera línea”. Es Spring Cloud Gateway (reactivo): no usa SecurityFilterChain servlet sino un GlobalFilter.

GatewayAuthFilter (GlobalFilter, order = -1)

Corre antes del ruteo. Hace tres cosas:

- 1) LIMPIEZA - de TODA request entrante borra los headers que sólo pueden venir del propio gateway/servicios:
X-Internal-Token, X-Auth-Username, X-Auth-Uid, X-Auth-Rol
-> un cliente no puede spoofearlos para hacerse pasar por otro servicio.
- 2) VALIDACIÓN - para /api/** (excepto /api/auth/**):
falta "Authorization: Bearer" 401 "Falta el token..."
jwtValidator.validar(token) == vacío ... 401 "Token inválido o expirado"
Corta acá, sin molestar a los servicios.
- 3) PROPAGACIÓN - si el token es válido, reinyecta hacia adentro:
X-Auth-Username = sub ; X-Auth-Uid = uid ; X-Auth-Rol = rol
(por si un servicio quiere la identidad sin re-parsear el JWT)

/api/auth/ queda fuera de la exigencia de token (login, refresh, etc. son públicos). Los assets del front y /uploads/ tampoco pasan por la rama protegida.

JwtValidator (@Component)

Validación mínima en el gateway: **misma clave pública RSA y mismo issuer** que los servicios (app.jwt.public-key-location, app.jwt.issuer). Devuelve Optional<Claims>: vacío ante cualquier error. No confía ciegamente: cada servicio vuelve a validar.

application.yml — CORS y ruteo

`globalcors → allowed-origin-patterns`	<code>\${GATEWAY_ALLOWED_ORIGINS:*</code> . En dev: * (cualquier origen; sirve el React desde el gateway en cualquier puerto sin pelear con CORS). En prod se OBLIGA a pasar la URL del frontend (Vercel).
`allowed-methods`	GET, POST, PUT, DELETE, OPTIONS
`allowed-headers`	Authorization, Content-Type
`allow-credentials`	false — la API usa JWT en header, no cookies; aceptar cualquier origen no expone nada.
`routes`	<code>/api/auth/ → auth · /api/usuarios/ → usuarios · /api/juegos/ → juegos · /api/secciones/ → secciones · /api/compras/ → compras · /uploads/ → catalogo</code>
`app.jwt.public-key-location / issuer`	La clave pública (classpath demo o archivo montado) y el issuer que exige JwtValidator.
`management...include`	health, info, gateway — expuestos por Actuator.

3 · auth-service — SecurityConfig

services/auth-service/.../security/SecurityConfig.java

Dos “puertas”: `/api/ stateless con JWT propio`, y `/internal/` detrás de `InternalTokenFilter`. El login con Google necesita sesión HTTP para su baile de redirects, por eso la sesión es `IF_REQUIRED` y no `STATELESS`.

El SecurityFilterChain, línea por línea

<code>` .cors(disable)`</code>	auth-service NO valida Origin. El único que habla con navegadores es el gateway. A auth-service sólo le llegan: llamadas server-to-server del gateway, el proxy de Vite en dev, y los redirects top-level de Google — ninguno es CORS. Si validara Origin, rechazaría lo que el gateway le reenvía → 403.
<code>` .csrf(disable)`</code>	API stateless con token en header, no cookies de sesión → CSRF no aplica. (OAuth2 usa su propio state.)
<code>` .formLogin/.httpBasic(disable)`</code>	No hay formulario de Spring ni Basic Auth. El login es un endpoint REST propio que devuelve JWT.
<code>` .sessionManagement(IF_REQUIRED)`</code>	No crea sesión salvo que algo la pida. Sólo el flujo oauth2Login la necesita (guardar el state entre el redirect a Google y la vuelta).
<code>` authorizeHttpRequests →`</code>	se evalúan EN ORDEN; gana la primera que matchea:
<code>` /actuator/health/`</code>	permitAll — healthcheck de Docker.
<code>` /internal/`</code>	hasRole(“INTERNAL”) — sólo lo alcanza quien pasó el InternalTokenFilter.
<code>` /api/auth/login,/refresh,/logout,/me`</code>	permitAll — puerta de entrada: pedir token, renovarlo, cerrarlo y resolver “quién soy” (auth → usuarios).
<code>` /oauth2/, /login/oauth2/`</code>	permitAll — endpoints internos del flujo OAuth2 de Spring (arranque del redirect y callback de Google).
<code>` /api/auth/2fa/`</code>	authenticated — activar / desactivar / consultar el 2FA requiere estar logueado.
<code>` anyRequest()`</code>	authenticated — todo lo demás exige JWT.
<code>` .oauth2Login(successHandler=...)`</code>	Habilita “Continuar con Google”. Al terminar OK, en vez de crear sesión web, el handler emite el par de tokens y redirige al frontend.
<code>` .exceptionHandling(...)`</code>	authenticationEntryPoint → 401 JSON {“mensaje”:“No autenticado”}. accessDeniedHandler → 403 JSON. (Sin esto Spring devolvería HTML.)
<code>` .addFilterBefore(InternalTokenFilter, UsernamePasswordAuthFilter)`</code>	Mete el filtro de token interno antes del punto donde Spring autenticaría por usuario/clave.
<code>` .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthFilter)`</code>	Igual para el filtro de JWT de common-security.

Beans de autenticación

<code>` @Bean PasswordEncoder`</code>	BCryptPasswordEncoder — hash de contraseñas (con salt, lento a propósito). Se usa al crear credenciales y al comparar en el login.
<code>` @Bean DaoAuthenticationProvider`</code>	Ata el UserDetailsService (busca la credencial por username) con el PasswordEncoder (compara el hash). Es quien realmente valida usuario+clave.
<code>` @Bean AuthenticationManager`</code>	Un ProviderManager explícito (no un lambda) para que se PUBLIQUEN los AuthenticationSuccess/FailureEvent. De esos eventos vive el LoginAttemptListener (lockout).

CredentialUserDetailsService · CredentialUserDetails

Adaptador Credential → UserDetails para que el DaoAuthenticationProvider pueda comparar el hash y aplicar el bloqueo.

<code>` loadUserByUsername`</code>	Busca la credencial; si no existe → UsernameNotFoundException.
<code>` getPassword()`</code>	Devuelve el hash BCrypt guardado.
<code>` isAccountNonLocked()`</code>	false mientras bloqueadoHasta esté en el futuro → el lockout corta el login sin comparar la clave.

<code>`isEnabled()`</code>	SIEMPRE true a propósito: el flag “habilitado” del usuario NO vive en auth-service; lo chequea AuthService después, contra usuarios-service.
----------------------------	--

LoginAttemptListener (@EventListener)

Bloqueo automático por intentos fallidos. Escucha los eventos del AuthenticationManager:

```
onFailure (AbstractAuthenticationFailureEvent):
    intentosFallidos++
    si intentosFallidos >= 5:
        bloqueadoHasta = now + 15 min ; intentosFallidos = 0

onSuccess (AuthenticationSuccessEvent):
    intentosFallidos = 0 ; bloqueadoHasta = null          # reset
    (ignora el login de Google: usa otro tipo de principal)
```

TotpService — 2FA (RFC 6238)

TOTP implementado a mano (mismo algoritmo que Google Authenticator / Authy). No es config de HttpSecurity pero es parte del stack de auth.

<code>`generarSecreto()`</code>	20 bytes aleatorios (SecureRandom) en Base32.
<code>`generarUri(secreto,cuenta)`</code>	otpauth://totp/GameStore:usuario?secret=...&issuer=GameStore&algorithm=SHA1&digits=6&period=30 — lo que se pasa a QR.
<code>`validar(secreto,codigo)`</code>	HMAC-SHA1 sobre el contador de 30s, “dynamic truncation” a 6 dígitos. Ventana ± 1 (tolera 30s de desfase de reloj).

OAuth2ClientConfig — Google opcional

Registra el cliente OAuth2 de Google **a mano** para que sea opcional de verdad: si app.oauth2.google.client-id/secret están vacíos, el ClientRegistrationRepository devuelve null para todo, la app arranca igual y el botón “Continuar con Google” queda inerte. Si están, arma la registración con CommonOAuth2Provider.GOOGLE.

OAuth2LoginSuccessHandler

Qué pasa cuando Google confirma la identidad:

```
1) del OAuth2User saca email y name
2) usuariosClient.buscarOCrearGoogle(email, name) # usuarios decide el rol -> COMPRADOR
3) credentialService.asegurarParaGoogle(username, email) # credencial local sin password
4) emite accessToken (JWT) + refreshToken
5) redirige a {frontend}/#/oauth-callback?token=...&refreshToken=...
```

4 · usuarios-service — SecurityConfig

services/usuarios-service/.../config/SecurityConfig.java

100% **stateless**. Cada request trae un JWT firmado por auth-service, o el header X-Internal-Token si es una llamada de otro servicio.

`@EnableWebSecurity`	Activa la cadena de filtros de Spring Security.
`@EnableMethodSecurity(prePostEnabled=true)`	Habilita @PreAuthorize en los controllers.
` .csrf/.formLogin/.httpBasic(disable)`	Igual que auth: API con token, sin formularios ni cookies.
` .sessionManagement(STATELESS)`	Nunca crea ni usa HttpSession. Cada request se autentica sola con su token.
` authorizeHttpRequests →`	
` /actuator/health/`	permitAll
` /internal/`	hasRole("INTERNAL") — p. ej. auth-service pidiendo rol/estado en el login, o alta de credencial.
` /api/usuarios/, /api/admin/`	hasRole("ADMIN") — ABM de usuarios y estadísticas: sólo administradores.
` anyRequest()`	authenticated
` .exceptionHandling(...)`	401 / 403 en JSON, igual que auth-service.
` .addFilterBefore x2`	Los dos filtros de common-security, antes del punto de autenticación usuario/clave.

@PreAuthorize en controllers: UsuarioController y AdminStatsController llevan @PreAuthorize("hasRole('ADMIN')") a nivel de clase — doble candado sobre la regla de URL (defensa en profundidad).

5 · catalogo-service — SecurityConfig

services/catalogo-service/.../config/SecurityConfig.java

Stateless. Mismas reglas de rol que el monolito, pero validando JWT en vez de sesión. La pertenencia (“¿es tu juego?”) no se puede expresar por URL → se revisa en el servicio.

`@EnableMethodSecurity(prePostEnabled=true)`	Habilita @PreAuthorize.
`.csrf().formLogin().httpBasic(disable) · STATELESS`	Idéntico a usuarios-service.
`.authorizeHttpRequests →`	ORDEN importa; gana la primera coincidencia:
`. /actuator/health/`	permitAll
`. /uploads/`	permitAll — las portadas de los juegos son públicas (imágenes estáticas).
`. /internal/`	hasRole(“INTERNAL”) — conteos por vendedor / stats para admin.
`. POST /api/juegos/*/comprar`	hasRole(“COMPRADOR”) — comprar (ficticio) es exclusivo del comprador. Va ANTES de la regla general de POST /api/juegos/.
`. GET /api/compras`	hasRole(“COMPRADOR”) — “Mis compras” del usuario actual.
`. POST /api/juegos, POST /api/juegos/`	hasAnyRole(“VENDEDOR”, “ADMIN”) — publicar juego, reponer stock (/ {id}/stock).
`. PUT /api/juegos/`	hasAnyRole(“VENDEDOR”, “ADMIN”) — editar juego.
`. DELETE /api/juegos/`	hasAnyRole(“VENDEDOR”, “ADMIN”) — borrar juego.
`. POST /api/secciones`	hasRole(“ADMIN”) — crear categoría.
`. DELETE /api/secciones/`	hasRole(“ADMIN”) — borrar categoría.
`. /api/ · anyRequest()`	authenticated — listar juegos/secciones: cualquier usuario logueado (incluido COMPRADOR, en modo lectura).
`.exceptionHandling(...)`	401 / 403 en JSON.
`.addFilterBefore x2`	Los dos filtros de common-security.

Autorización “a nivel de dato” (en el servicio, no en la URL)

- **JuegoController:** @PreAuthorize(hasAnyRole VENDEDOR,ADMIN) en crear/editar/borrar/stock, y además verificarPropietarioOAdmin() → JuegoService.puedeGestionar(): el ADMIN modera cualquier juego; el VENDEDOR sólo los suyos (compara vendedorUsername contra el username del JWT). Si no, AccessDeniedException → 403.
- **CompraController:** @PreAuthorize(hasRole COMPRADOR) en comprar y en “mis compras”; el listado se filtra por el username del token, no por un parámetro que el cliente pueda cambiar.
- **SeccionController:** @PreAuthorize(hasRole ADMIN) en crear/borrar.

6 · Tabla resumen — qué protege cada cosa

Mecanismo	Dónde	Contra qué protege
Firma RS256 (privada sólo en auth)	JwtService	Que otro servicio (o un atacante) emita tokens válidos. Sólo auth firma.
verifyWith(publicKey) + requireIssuer	JwtService / JwtValidator	Tokens falsos, alterados, de otro emisor o expirados.
Validación en el gateway (1ª línea)	GatewayAuthFilter	Que requests sin token lleguen siquiera a los servicios.
Re-validación en cada servicio	JwtAuthenticationFilter	Que saltarse el gateway (acceso directo) baste. Defensa en profundidad.
Limpieza de X-Internal-Token / X-Auth-*	GatewayAuthFilter	Que un cliente se haga pasar por un servicio interno o por otro usuario.
InternalTokenFilter (X-Internal-Token)	auth / usuarios / catalogo	Que el navegador llame a los endpoints /internal/.
authorizeHttpRequests + hasRole	cada SecurityConfig	Que un rol acceda a operaciones de otro (COMPRADOR publicando, etc.).
@PreAuthorize en controllers	usuarios / catalogo	Doble candado si una regla de URL se afloja por error.
puedeGestionar() (username del JWT)	JuegoService	Que un VENDEDOR edite/borre juegos de otro vendedor.
BCrypt	auth-service	Que un dump de la BD revele contraseñas.
Lockout 5 / 15 min	LoginAttemptListener	Fuerza bruta sobre una cuenta.
2FA TOTP	TotpService	Login con contraseña robada.
Refresh token en BD (revocable)	auth-service	Que un logout / baja no pueda cortar la sesión (un JWT suelto no se invalida).
STATELESS	usuarios / catalogo	Fijación de sesión y sincronizar sesiones entre servicios.
CORS sólo en el gateway	application.yml	Que un sitio de terceros llame a la API desde el navegador de la víctima.

7 · Flujo de una request

a) Login con usuario y contraseña

```
navegador --POST /api/auth/login--> gateway
  (/api/auth/** está exento de token en el gateway)
gateway --> auth-service /api/auth/login (permitAll)
  AuthenticationManager -> DaoAuthenticationProvider:
    CredentialUserDetailsService.loadUserByUsername
    isAccountNonLocked? (lockout)
    BCrypt.matches(clave, hash)
    -> éxito/fallo publica evento -> LoginAttemptListener
  si 2FA habilitado y falta código -> 401 "requiere2fa"
  AuthService consulta usuarios-service: ¿rol? ¿habilitado?
  JwtService.generar(uid, username, rol) # firma RS256
  RefreshTokenService.crear(username) # fila en authdb
  -> { token, refreshToken, usuario }
```

b) Request autenticada (p. ej. comprar un juego)

```
navegador --POST /api/juegos/9/comprar (Authorization: Bearer <jwt>)--> gateway
GatewayAuthFilter:
  borra X-Internal-Token / X-Auth-*
  /api/** y no /api/auth/** -> exige token
  JwtValidator.validar(jwt) (firma + issuer) ok
  reinyecta X-Auth-Username / X-Auth-Uid / X-Auth-Rol
  rutea a catalogo-service
catalogo-service:
  JwtAuthenticationFilter re-valida el JWT -> SecurityContext con ROLE_COMPRAADOR
  regla: POST /api/juegos/*/comprar -> hasRole("COMPRADOR") ok
  @PreAuthorize("hasRole('COMPRADOR')") ok
  CompraService: ¿ya lo compró? ¿hay stock? -> descuenta con lock de fila
  -> 201 { compra }
```

Generado a partir del código en services/. Claves JWT del repo = sólo demo (regenerar con keys/generate-keys.sh para producción; no commitear la privada).